malloc 之后内存去哪了：为何程序越跑越占内存

嵌入式 Linux 手记　2026-10-30　公众号文章 · 原链接 mp.weixin.qq.com

先猜答案。这两个现象，哪一个出了 bug?

```
void *p = malloc(1);          /* 只想要 1 字节 */
/* 内存水位为什么掉了 24 个字节往上?  */

void *q = malloc(1 << 20);    /* 1MB，用完就 free */
/* 反复一晚上，RSS 为什么只涨不降?  */
```

现象一：程序反复 `malloc` 1MB、用完 `free`，循环跑一晚上，RSS（常驻内存）涨了 1MB 左右就不再变化，而且一分都不回落。现象二：`malloc(1)` 只想要 1 字节，内存水位却掉了 24 个字节往上。

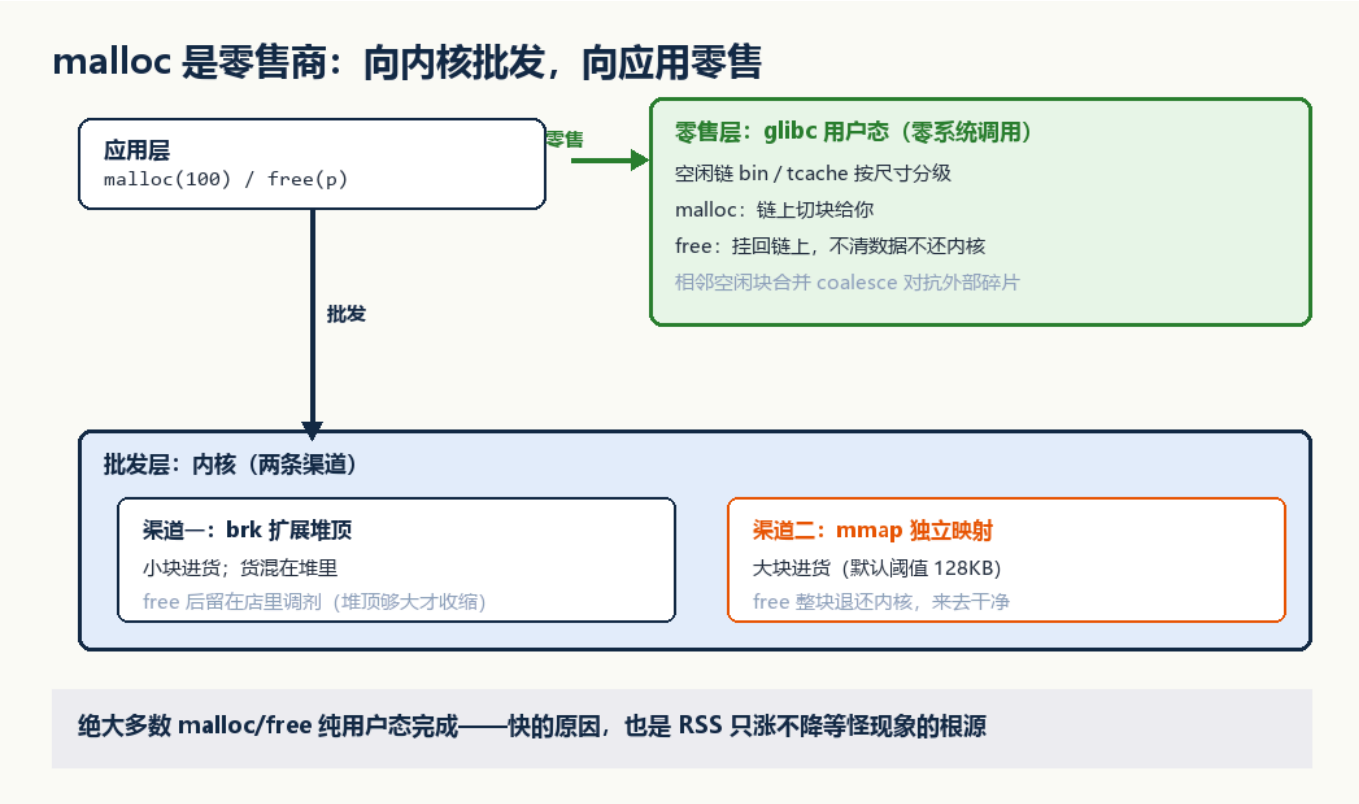
答案：两个都不是 bug，都是 glibc 的仓库管理制度在正常运转。现象一的"泄漏"多半是假象，现象二的 24 字节是仓库收的管理费。这一篇进 glibc 的批发仓库，把 `malloc` 之后内存的去向、`free` 的真实动作、RSS 只涨不降的账本全部对清。篇 8 从 UB 视角讲过"free 之后那块内存里有什么"，这一篇讲它的管理制度：同一块内存，两次对账。

一、malloc 是零售商，不是搬运工

很多初学者把 `malloc` 想象成"向内核要内存的窗口"，每次调用都直达内核。真实结构是两层：`malloc` 是库函数，不是系统调用，它自己开了一家零售店：向内核批发，向应用零售。

批发走两条渠道。小块进货用 `brk` 系统调用扩展堆顶，堆是一段连续区域，堆顶指针往高处推，新批内存就到手；大块进货走 `mmap`，直接映射一段独立区域，glibc 的分界线默认 128KB（可用 `mallopt(M_MMAP_THRESHOLD, ...)` 调整）。为什么大块另开渠道？`mmap` 的货 `free` 时可以整块退还内核，来去干净；`brk` 的货混在堆里，只能留在店里慢慢调剂。还有一个反直觉的细节：这个 128KB 阈值是动态的，`free` 一块 mmap 大块时，glibc 会把阈值自动上调到该块大小（上限 32MB），配对的 trim 阈值同步翻倍。于是"反复 malloc/free 同样的 1MB"从第二次起就改走 brk 复用，RSS 涨约 1MB 便封顶，这正是现象一"涨了不回落、但也不再涨"的完整答案。篇 8 讲过"大块 free 后再访问必崩、小块 free 后读旧值多半不崩"的差异，根源就是这条分界线。

零售发生在库内部。你 `malloc(100)` 时，glibc 不去打扰内核，它在自家的空闲内存链里找一块够大的零货切给你； `free` 时也不退还内核，货挂回空闲链，等下一次分配复用。绝大多数 `malloc` / `free` 纯用户态完成，零系统调用，这正是它快的原因，也是后面一切"怪现象"的根源。



图：批发零售两层结构。上层零系统调用，下层两条渠道（brk/mmap）按 128KB 分界；篇 8 的大块 free 后再访问必崩、小块不崩，根源就是这条分界线。

二、chunk：仓库的库存单位

空闲链上的每一块货，都有自己的包装：chunk。你拿到手的是货本体，但每块货前面都带着一个头部（chunk header），记录这块的自身大小和一组标志位（如 `PREV_INUSE` ，前一块是否在使用）。64 位上 chunk 按 16 字节对齐，头部占去 8~16 字节，这就是 `malloc(1)` 实际占 24 个字节往上的原因：你要的 1 字节装进货箱，箱子前面的标签、对齐垫出来的空隙，全都算你的账。这部分浪费叫内部碎片：分给你的块比你需要的大，差值在块内部蒸发。

这个头部还顺手解释了一个 API 设计： `free(p)` 为什么不用传大小？因为大小就写在 `p` 往前数几个字节的头部里， `free` 一看便知。释放的信息从来不用调用方操心，仓库自己记账。

三、free 之后，内存去哪了

`free` 的真实动作是：把这块 chunk 标记为空闲，挂回空闲链（glibc 里按尺寸分级，叫 bin，最快的快速通道叫 tcache），然后——没了。不请内存，不清数据，不碰你的指针。三个常见"怪现象"全部由此导出：



图：chunk 头部展开与 free 后的三个现象。管理费、箱子里没清的旧值、留在店里的货，一图对清。

现象一，`free` 之后读旧值还在。货退回仓库，箱子里上一位客户的东西还没清走，你凑近看当然还在，但此时访问它是 UB（篇 8 定过案），仓库随时可能把箱子发给下一位。

现象二，RSS 只涨不降。程序反复分配释放，空闲链越拉越长，可是链上的块零散分布在堆的各处，堆顶（top chunk）不够大就不收缩、不还货给内核——RSS 自然只涨不降。注意这不是泄漏：内存都在空闲链里，下次分配还能用，只是留在店里了。真正的泄漏是货发出去永远不回来（指针丢了，`free` 不了），两者的鉴别要靠工具（valgrind、ASan 的 leak 检查）看"分配了未释放"的清单，而不是只盯 RSS。

现象三，总空闲量够，却分不出连续大块。空闲链上东一块西一块，单块都够不着你要的尺寸，这是外部碎片：浪费发生在块与块之间。长期运行的程序最怕它，救急手段是 `malloc_trim` 主动催店，治本靠分配策略。

相邻的空闲块会合并（coalesce）成大块再复用，这是对抗外部碎片的主力机制：两块小零货拼成一块大货，大请求又有得卖了。合并的判断依据正是头部里的 `PREV_INUSE` 标志：前一块是否空闲，看标志位就知道，不用遍历。

四、多线程抢仓库：arena 的账

多线程程序还要多算一笔锁的账。早期 `malloc` 全店一把锁，四个线程同时分配就得排队，锁竞争让"零系统调用的快"打了折扣。glibc 的 ptmalloc 于是按线程分区（arena）：每个线程倾向使用自己的分区，各逛各的货架，锁冲突大幅下降。工业界另有 jemalloc、tcmalloc 等实现，思路相近而策略各异（更细的尺寸分级、更激进的线程缓存），高并发服务换分配器换出两位数百分比的性能，是真实存在的优化手段。

五、嵌入式的那本账

这套批发零售制度在桌面是效率神器，搬到单片机却水土不服：总 RAM 几十 KB 的系统经不起外部碎片的慢性侵蚀：总空闲量够、连续块不够，设备跑得越久越容易死。嵌入式对策是换一套制度：静态内存池（定长数组 + 自研分配器）、FreeRTOS 的 `heap_4`（首次适应 + 合并）、定长对象池；核心纪律是避免高频变长的 `malloc/free` 抖动。为什么 MCU 工程干脆禁用 `malloc`？MISRA 怎么说、FreeRTOS 五套官方分配器怎么选？这是下一篇的主场。

写在最后

1. `malloc` 是库函数不是系统调用：向内核批发（小块 `brk` 扩堆顶、大块 128KB 走 `mmap`），向应用零售（空闲链用户态完成，零系统调用）。
2. chunk 头部记录大小与标志位（PREV_INUSE），16 字节对齐，`malloc(1)` 实占 24+ 字节是管理费（内部碎片）；`free` 不带 size 的秘密也埋在头部。
3. `free` 的动作是挂回空闲链：不清数据、不还内核。三个现象一次对清——旧值还在（箱子没清，UB）、RSS 只涨不降（零散空闲块不收缩，非泄漏）、总量够分不出（外部碎片）。
4. 相邻空闲块合并（coalesce）是对抗外部碎片的主力，判断依据就是头部标志位；多线程按 arena 分区降锁竞争，jemalloc/tcmalloc 是同思路的工业实现。
5. 桌面效率神器，嵌入式慢性毒药：总 RAM 几十 KB 的系统换静态池与定长对象池，下篇专讲。

堆的账本翻到下一页：为什么安全认证直接把 malloc 写进禁用清单？下一篇《为什么 MCU 工程不写 malloc：静态内存池》，MISRA 红线、FreeRTOS 五套分配器与确定性分配的 O(1) 世界。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。